# Studio Nx

Backend(FastAPI/Python) 38 — save_profile_image()

2026-10-07

# Backend(FastAPI/Python) 38 — `save_profile_image()` (+ `image_extension()`)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/services/user_service.py` |
| 작업 | P1-08 회원 정보 — 프로필 사진 올리기 · PRD 01 FR-106 (jpg/png/webp, 2MB) |
| 작성 | Claude (AI) · 2026-10-07 · 사용자 검토 |
| 이전 · 다음 | 37 `update_me()` · 39 `delete_profile_image()` |

## 1. 하는 일

올라온 사진을 검사하고 `storage/profiles/{무작위 32자}.{확장자}` 에 저장한 뒤, 회원의 `profile_image_path` 를 바꾼다. 예전 사진 파일은 지운다.

| 올라온 파일 | 결과 |
|---|---|
| JPG (1.2MB) | 저장 → `profiles/3f9c….jpg`, 응답 사진 주소도 바뀜 |
| 브라우저는 `image/png` 라고 했지만 내용은 JPG | 저장 — 확장자는 **내용대로** `.jpg` |
| GIF | 415 `UNSUPPORTED_FILE_TYPE` "Use a JPG, PNG or WebP image." |
| `image/png` 라면서 내용은 HTML · 빈 파일 | 415 (같음) |
| 2MB 를 1바이트라도 넘음 | 413 `FILE_TOO_LARGE` "Images must be 2 MB or smaller." |
| 정확히 2MB | 저장 |
| 이미 사진이 있음 | 새 사진 저장 → 예전 파일 삭제 |

오류일 때는 파일이 하나도 남지 않고, DB 도 그대로다.

## 2. 언제 쓰나

| 쓰는 곳 | |
|---|---|
| `POST /users/me/profile-image` (42) | `multipart/form-data` 의 `file` 칸 → 이 함수 → 36 응답 |
| 화면 | userInfo.html 의 Upload photo (P1-10) — 화면도 같은 종류 · 크기를 먼저 검사함 (`PHOTO_TYPES` · `PHOTO_MAX_BYTES`) |

용량: 프로필 사진은 저장 한도 5GB 사용량에 **넣지 않음** (2026-10-07 사용자 결정) → `storage_used_bytes` 를 건드리지 않는다.

## 3. 코드

**import** (파일 맨 위)

```
from pathlib import Path, PurePosixPath
from uuid import uuid4
```

```
from fastapi import UploadFile
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
from app.models import User
from app.schemas.user import MeResponse, MeUpdateRequest
from app.services.auth_service import check_nickname
```

**`uuid4` · `get_settings` 를 이번에 추가했다.**

### 도우미 — 내용으로 종류 알아내기

```
def image_extension(data: bytes) -> str | None:
    if data.startswith(b"\xff\xd8\xff"):
        return ".jpg"
    if data.startswith(b"\x89PNG\r\n\x1a\n"):
        return ".png"
    if data[:4] == b"RIFF" and data[8:12] == b"WEBP":
        return ".webp"
    return None
```

### 함수

```
async def save_profile_image(db: Session, user: User, file: UploadFile) -> User:
    if file.content_type not in PHOTO_TYPES:
        raise AppError(ErrorCode.UNSUPPORTED_FILE_TYPE, 415, "Use a JPG, PNG or WebP image.")
    data = await file.read(PHOTO_MAX_BYTES + 1)
    if len(data) > PHOTO_MAX_BYTES:
        raise AppError(ErrorCode.FILE_TOO_LARGE, 413, "Images must be 2 MB or smaller.")
    extension = image_extension(data)
    if extension is None:
        raise AppError(ErrorCode.UNSUPPORTED_FILE_TYPE, 415, "Use a JPG, PNG or WebP image.")

    storage_dir = get_settings().storage_dir
    relative = f"{PROFILE_DIR}/{uuid4().hex}{extension}"
    new_file = storage_dir / relative
    new_file.parent.mkdir(parents=True, exist_ok=True)
    new_file.write_bytes(data)

    old_relative = user.profile_image_path
    user.profile_image_path = relative
    try:
        db.commit()
    except Exception:
        db.rollback()
        new_file.unlink(missing_ok=True)
        raise
    if old_relative:
        (storage_dir / old_relative).unlink(missing_ok=True)
    db.refresh(user)
    return user
```

### 문법 풀이

| import 문법 | 뜻 |
|---|---|
| `from uuid import uuid4` | 무작위 고유 번호. `.hex` = 16진수 32자 |
| `from app.core.config import get_settings` | 설정(`.env`) — `storage_dir` = `studionx/storage` (git 에서 제외된 폴더) |

| 식 | 뜻 |
|---|---|
| `async def` · `await file.read(n)` | 파일을 **최대 n 바이트**만 읽음. 업로드는 네트워크를 기다리므로 FastAPI 의 `UploadFile` 이 비동기 |
| `b"\xff\xd8\xff"` | 바이트 글자 — `\xff` = 16진수 FF 한 바이트 |
| `data.startswith(…)` · `data[8:12]` | 앞부분이 같은지 · 8 ~ 11번째 바이트 |
| `storage_dir / relative` | `Path` 의 `/` = 경로 잇기 |
| `mkdir(parents=True, exist_ok=True)` | 중간 폴더까지 만들고, 이미 있어도 오류 없음 |
| `unlink(missing_ok=True)` | 파일 삭제, 없어도 오류 없음 |
| `except Exception: … raise` | 정리만 하고 **같은 오류를 다시** 던짐 (삼키지 않음) |

## 4. 흐름

1. 브라우저가 알려 준 종류 확인 → 아니면 415.
2. 2MB + 1바이트까지만 읽기 → 넘으면 413.
3. 앞 바이트로 실제 종류 확인 → 아니면 415. 확장자는 여기서 정함.
4. 무작위 이름으로 파일 쓰기.
5. DB 에 새 경로 저장 → 실패하면 새 파일을 지우고 오류 그대로.
6. 성공했으면 예전 사진 파일을 지우고, DB 에서 다시 읽어 돌려줌.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 종류를 **두 번** 확인 (알려 준 값 + 내용) | `content_type` 은 브라우저(또는 공격자)가 마음대로 보낼 수 있다. HTML 을 `image/png` 라고 보내 저장시키면, 나중에 그 주소를 연 사람의 브라우저에서 스크립트가 돌 위험이 있다 |
| 확장자를 **내용**으로 정함 | 파일을 돌려줄 때(40 · 43) 확장자로 종류를 정한다. 내용과 확장자가 맞아야 한다 |
| `PHOTO_MAX_BYTES + 1` 만 읽음 | 10GB 파일을 올려도 2MB 남짓만 메모리에 올라온다. 1바이트를 더 읽어야 "딱 2MB" 와 "넘음" 을 구분할 수 있다 |
| 저장 → DB → 예전 파일 삭제 순서 (템플릿은 "예전 파일을 지우고 → 저장") | DB 저장이 실패해도 예전 사진이 남는다. 템플릿 순서면 실패할 때 사진이 사라진다. 시험으로 확인 |
| 이름은 무작위 32자 (회원 번호 없음) | 35 의 사용자 결정. 같은 회원이 사진을 바꾸면 이름도 바뀌어 캐시 문제도 없다 |
| `except Exception` (넓게) | DB 연결이 끊기는 등 어떤 오류든 새 파일을 남기지 않으려고. 오류는 다시 던져 06 의 500 처리로 간다 |
| 그림 전체를 열어 보지는 않음 | 앞 바이트 검사로 종류는 확실히 가린다. 이미지 라이브러리(Pillow)로 다시 저장하면 더 안전하지만 패키지가 하나 는다 — 필요하면 나중에 |

| 선택 | 이유 |
|---|---|
| 사진 파일 응답에 `nosniff` | 43 에서 `X-Content-Type-Options: nosniff` 를 붙여, 브라우저가 내용을 보고 종류를 바꿔 추측하지 않게 할 예정 |

## 6. 시험 (`tests/test_user_service.py` 에 17개 추가 · 모두 39개, 실제 MySQL)

사진은 시험용 임시 폴더(`tmp_path`)에 저장 — 진짜 `storage/` 는 건드리지 않음.

| 시험 | 확인한 것 |
|---|---|
| `image_extension` 7가지 | jpg · png · webp / gif · HTML · WAV(RIFF 지만 WEBP 아님) · 빈 파일 → `None` |
| JPG 저장 | 경로 `profiles/{32자}.jpg`, 파일 내용 같음, 파일 1개, 응답 주소, DB 경로, `storage_used_bytes` 0 그대로 |
| png 라면서 jpg | `.jpg` |
| GIF · HTML · 빈 파일 · 2MB 초과 | 415 · 415 · 415 · 413, 파일 0개, 경로 그대로 |
| 정확히 2MB | 저장 |
| 사진 바꾸기 | 새 이름, 예전 파일 삭제, 파일 1개 |
| 옛 파일이 이미 없음 | 오류 없이 저장 |
| DB 저장 실패 | 새 파일 삭제, 예전 사진 파일 · 내용 그대로, 오류 그대로 |

전체 시험 273개 통과, `ruff` 검사 통과.